SIMATS ENGINEERING
ASSESSMENT TOOL 1

COURSE NAME: Artificial Intelligence COURSE CODE: CSA17

COURSE OUTCOME COVERED

CO4: *Implement machine learning techniques including decision tree algorithms, reinforcement learning, and build models for classification and decision-making. (BL3)*

Assessment Tool Weightage: 40%

Duration : 45 Minutes
QUESTIONS: 2 **Total Marks:20**

Annexure A

INTEGRATED EXPERIMENTS

Code of Conduct: **G Hema chowdary**

I _____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

SIMATS ENGINEERING – CSA17 ARTIFICIAL INTELLIGENCE

CO4 ASSESSMENT TOOL 1 – COMPLETE SOLUTIONS

The assessment contains two integrated experiments: Decision Tree Classification and Reinforcement Learning – Q-Learning. ■filecite■turn0file0■L19-L36■

Experiment 1: Decision Tree Classification

Objective: Implement and evaluate a Decision Tree classifier on a dataset such as Iris/Play-Tennis.

(a) Load, preprocess and split the dataset

```
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
import pandas as pd

iris = load_iris()
X = pd.DataFrame(iris.data, columns=iris.feature_names)
y = pd.Series(iris.target, name="target")

data = X.copy()
data["target"] = y
print(data.head())
print(data.dtypes)

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.20, random_state=42, stratify=y
)
```

Explanation: The Iris dataset contains numerical features, so no categorical encoding is needed. The dataset is split into 80% training and 20% testing data. Missing-value handling can be checked with *data.isnull().sum()*.

(b) Build the Decision Tree and identify the root node

```
from sklearn.tree import DecisionTreeClassifier, export_text

model = DecisionTreeClassifier(
    criterion="gini", max_depth=3, random_state=42
)
model.fit(X_train, y_train)

print(export_text(model, feature_names=list(X.columns)))
print("Root feature:",
      X.columns[model.tree_.feature[0]])
```

Root-node justification: The root is the feature selected by the tree for the first split because it gives the greatest impurity reduction according to the chosen Gini criterion. For the Iris dataset, petal-related features commonly provide the strongest first split. The exact root should be read from the printed tree for the selected parameters.

(c) Model evaluation

```
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report

y_pred = model.predict(X_test)

print("Accuracy:", accuracy_score(y_test, y_pred))
print("Confusion Matrix:")
print(confusion_matrix(y_test, y_pred))
print(classification_report(
    y_test, y_pred, target_names=iris.target_names
))
```

Performance: The Decision Tree normally gives high accuracy on Iris because the classes are well separated by petal measurements. The confusion matrix identifies correctly and incorrectly classified samples. To list at least two actual misclassified instances without inventing results, use the following code.

```
wrong = X_test[y_test != y_pred].copy()
wrong["Actual"] = y_test[y_test != y_pred].map(
    dict(enumerate(iris.target_names))
)
wrong["Predicted"] = pd.Series(
    y_pred[y_test.to_numpy() != y_pred],
    index=wrong.index
).map(dict(enumerate(iris.target_names)))
print(wrong.head(2))
```

Experiment 2: Reinforcement Learning – Q-Learning

Objective: Implement Q-Learning for a 4×4 Grid World and observe the learned policy. Required actions are Up/Down/Left/Right, with +10 for the goal, –1 for each step and –5 for an obstacle. The task requires at least 100 episodes and Q-values at Episodes 1, 50 and 100. ■filecite■turn0file0■L29-L36■

(a) Environment, Q-table and hyperparameters

```
import numpy as np
import random

N = 4
start, goal, obstacle = (0,0), (3,3), (1,1)
actions = ["Up", "Down", "Left", "Right"]
moves = [(-1,0), (1,0), (0,-1), (0,1)]

alpha = 0.10
gamma = 0.90
epsilon = 0.20
Q = np.zeros((N, N, 4))

def step(state, action):
    r, c = state
    dr, dc = moves[action]
    nr, nc = r + dr, c + dc

    if nr < 0 or nr >= N or nc < 0 or nc >= N:
        return state, -1, False

    ns = (nr, nc)
    if ns == obstacle:
        return ns, -5, False
    if ns == goal:
        return ns, 10, True
    return ns, -1, False
```

(b) Q-Learning for 100 episodes

```
episode_rewards = []
snapshots = {}

for ep in range(1, 101):
    state = start
    total = 0

    for _ in range(100):
        if random.random() < epsilon:
            action = random.randrange(4)
        else:
            action = int(np.argmax(Q[state[0], state[1], :]))

        ns, reward, done = step(state, action)
        old = Q[state[0], state[1], action]
        best = np.max(Q[ns[0], ns[1], :])

        Q[state[0], state[1], action] = old + alpha * (
            reward + gamma * best - old
        )
        state = ns
        total += reward

        if done:
            break

    episode_rewards.append(total)

    if ep in [1, 50, 100]:
        snapshots[ep] = {
            "(0,0)": Q[0,0,:].copy(),
            "(2,2)": Q[2,2,:].copy()
        }

for ep in [1, 50, 100]:
    print("Episode", ep, snapshots[ep])
```

Q-table evolution: At Episode 1, values are near zero because the agent has little experience. By Episode 50, actions that repeatedly lead toward the goal acquire stronger Q-values. By Episode 100, the values become more stable and the best actions tend to form a route toward the goal while avoiding the obstacle.

(c) Final learned policy and reward plot

```
symbols = {"Up":"↑", "Down":"↓", "Left":"←", "Right":"→"}

for r in range(N):
    row = []
```

```
    for c in range(N):
        if (r,c) == goal:
            row.append("G")
        elif (r,c) == obstacle:
            row.append("X")
        else:
            a = int(np.argmax(Q[r,c,:]))
            row.append(symbols[actions[a]])
    print(row)
```

```
import matplotlib.pyplot as plt
plt.plot(episode_rewards)
plt.xlabel("Episode")
plt.ylabel("Cumulative Reward")
plt.title("Q-Learning: Reward per Episode")
plt.grid(True)
plt.show()
```

Convergence: The agent gradually learns which actions produce higher long-term rewards. As useful Q-values increase and the policy becomes stable, the cumulative reward generally improves. Some fluctuations can remain because ε-greedy exploration deliberately selects non-optimal actions.

Final Conclusion

The Decision Tree experiment demonstrates supervised classification using feature-based splitting and standard evaluation measures. The Q-Learning experiment demonstrates reinforcement learning through rewards, penalties and iterative policy improvement. These experiments address the CO4 outcome on machine-learning techniques, classification and decision-making. ■filecite■turn0file0■L5-L10■